

Every piece of technology in the system: what it is, why it was chosen, and what it replaces later

OCloud Solutions · state on 11 September 2026 · from requirements.txt, the imports, and the artefacts on disk

The stack is small on purpose. Python and its standard library carry most of the system; twelve third-party libraries do the rest; four of those are only needed to train models and never run in production. Every choice below is stated with the alternative it was chosen over.

1	14	12	0
language — Python 3.12	standard-library modules doing real work	third-party libraries at runtime or training	web frameworks, JS build steps, agent frameworks

The stack in one table

LAYER	TECHNOLOGY	IN THE POC	IN PRODUCTION
Language	Python 3.12	everything	same
Mailbox access	Gmail API via google-api-python-client + OAuth; IMAP via imaplib + app password; SMTP via smtplib	both doors	same, plus Graph, JMAP
Parsing	email, selectolax, email-reply-parser, icalendar, dateparser, zipfile	as listed	plus HeidelTime / duckling for dates, factur-x for invoices
Our models	transformers + torch (XLM-RoBERTa-base x2), lightgbm, scikit-learn, numpy	run on CPU	ONNX 8-bit, no PyTorch installed
Rented model	Gemini via google-genai	free tier	Claude or Gemini, by blind comparison
Storage	SQLite via sqlite3; JSON files; prompts.md	one file	Postgres + pgvector

Surfaces	<code>http.server</code> + plain HTML/CSS; terminal with ANSI colour	local browser and terminal	WhatsApp as the channel; the web view served properly
Training compute	Kaggle free T4 GPU; laptop CPU	minutes on Kaggle	a scheduled retrain per user, weekly
Evidence	<code>runlog.py</code> → <code>test_runs/</code> ; Chrome headless for PDFs	every proof run saved	same

- A Language and runtime
- B The standard library, module by module
- C Reaching the mailbox
- D Parsing mail
- E Models and machine learning
- F Storage
- G Surfaces: web and terminal
- H Secrets, configuration and flags
- I What is deliberately not used
- J The production swap list

A • Language and runtime

Python 3.12

WHAT

The one language in the project. Every script, every stage, every connector, the web server and the training code.

WHY

Three reasons that reinforce each other. The mail protocols, SQLite, an HTTP server and email parsing are all in the standard library, so the POC runs with almost nothing installed. The model ecosystem (`transformers` , `lightgbm` , `scikit-learn`) is Python-first, so training and serving share a language. And the people who will read and change this code read Python.

WHY 3.12 SPECIFICALLY

It is what is installed on the build machine. Nothing in the code needs a newer feature; `str | None` annotations and `match` -free code mean 3.10+ would run it.

WHY NOT OTHERWISE

A Node or Go service would need a separate Python process for the models anyway. Two languages for a proof of concept is two things that can disagree.

No framework, on purpose

There is no Flask, FastAPI, Django, LangChain, LlamaIndex or agent framework anywhere in the project. Flask and FastAPI were checked and are simply not installed; the web page was written on `http.server` instead so that the POC needs no install step to be looked at. Agent frameworks were rejected for a stronger reason: the safety of the system depends on the approval gate being ordinary Python that never sees email text, and on instruction text and email text never sharing a code path. A framework that routes everything through one model loop removes exactly that separation. Sir also specifically refrained from an MCP-server style design; the chat surface is a plain read loop with a multiple-choice question our code constructs.

B • The standard library, module by module

Fourteen modules from Python's own library carry more of the system than any third-party package. Listed with where each is used and why it was enough.

MODULE	USED IN	WHAT FOR, AND WHY THE STANDARD ONE
sqlite3	store.py, stage19_voice.py	The whole database. Zero setup, one file, transactional, and every query in the POC touches a few thousand rows. Opened with <code>check_same_thread=False</code> so the web page and its background watcher can share the file.
imaplib	connectors/imap.py	The IMAP door. Speaks the protocol over TLS on port 993. Its <code>.uid()</code> method is used for <i>every</i> command because its plain <code>fetch/search/store/copy</code> use sequence numbers, which shift. It does not quote arguments, so folder names with spaces are quoted by hand.
smtplib	connectors/imap.py	Sending through the IMAP door, on port 587 with STARTTLS, using the same app password. The sent copy is then APPENDED to the Sent folder over IMAP, because SMTP keeps no record.
email	stage02, stage05, both connectors, audit	Parsing raw RFC 822 bytes into headers and parts (<code>message_from_bytes</code> with the modern <code>default</code> policy), building outgoing messages (<code>EmailMessage</code>), and <code>make_msgid</code> / <code>formatdate</code> / <code>parsedate_to_datetime</code> . Handles encoded headers so a curly apostrophe in a subject survives.
http.server	web.py	The browser view. <code>BaseHTTPRequestHandler</code> with four routes. Enough for one user on localhost; not a production server and not pretending to be.
threading	web.py, connectors/gmail.py	The background agent thread in the web page (its own connection and its own database handle), and thread-local Google clients because Google's client is not thread-safe.
concurrent.futures	connectors/gmail.py	Six worker threads fetching envelopes in parallel through the API, inside the quota.
re	almost everywhere	Sensitive-subject patterns, identifier patterns for redaction, the chat's rule matcher, header parsing. Regular expressions do the structured work so models are not spent on it.
json	store, corrections, audits, model metadata	Small records on disk (<code>applied_labels.json</code> , <code>gate_model.json</code> , <code>user_sensitive.json</code>) and parsing model answers.
hashlib	stage13_approval.py	SHA-256 of the draft text. A typed yes is bound to that hash; editing the draft voids the yes.
zipfile	stage05_forms.py	Opening <code>.pkpass</code> wallet passes, which are ZIPs with a JSON file inside.

datetime	stages 7, 9, 16; store	All date arithmetic. Reminder lead times, business-hours checks, ISO dates for sorting. Never a model.
contextlib	scan.py	<code>redirect_stdout</code> to capture the pipeline's sixteen-stage trace during a backlog scan and log one line instead.
statistics · webbrowser · pathlib	limits test; web.py; everywhere	Medians over every message size; opening the browser on start; paths.

C · Reaching the mailbox

google-api-python-client + google-auth-oauthlib — the Gmail API door

WHAT

Google's official Python client for the Gmail REST API, and the OAuth 2 flow that gets a user's consent. `credentials.json` identifies our app to Google; `token.json` holds the user's grant and refresh token.

WHY

The richest access: push, native labels, threads, `history.list` ("what changed since X" for one quota unit), and the label-removal feed that makes corrections made inside Gmail visible.

HOW IT IS CONSTRAINED

Two scopes, `gmail.modify` and `gmail.send`. The full-access scope that allows permanent delete is never requested. Headers are fetched by name with `format=metadata`, so an envelope scan never downloads a body.

THE COST, AND WHY IT IS NOT THE ONLY DOOR

Google's CASA security audit (\$540–3,000 a year, repeated annually) before the OAuth screen can serve more than 100 users; 7-day token expiry until approved; 2–3 months of approval. A billing rule, not a technical one.

IMAP + an app password — the free door

WHAT

The protocol every mail server speaks, authenticated with a 16-character password Google issues from the account's security page. No OAuth screen, so none of the audit, expiry or user cap.

WHY IT IS NOT A DOWNGRADE ON GMAIL

Gmail's IMAP advertises `X-GM-EXT-1`, which adds `X-GM-RAW` (the full Gmail search language over IMAP, including `category:primary`), `X-GM-LABELS` (real labels, readable and writable) and `X-GM-THRID` (real thread ids). Measured: 45 of 47 audited capabilities work.

WHAT IT GENUINELY LACKS

Server push (IDLE exists but holds a socket per mailbox; we poll by highest UID every 20 s) and snooze. Everything else, including sending, is covered.

LIMITS, MEASURED

2,500 MB/day download cap: our whole design uses 2.3% on the first day. 15 simultaneous connections per account: we use one or two. Sending: 500 recipients/day on consumer Gmail.

SMTP — sending through the free door

`smtp.gmail.com:587`, STARTTLS, the same app password. Measured 4.9 s for a delivered message with the copy filed in Sent. It is called from one place only, the executor in stage 14, after the gate.

Not yet: Microsoft Graph, JMAP, Zoho

Each is one new connector file behind the same interface. Deferred because sir asked for Google only for now. Proton is impossible (end-to-end encrypted); iCloud was declined (no API, only a password route).

D • Parsing mail

LIBRARY	USED FOR	WHY THIS ONE	INSTEAD OF
<code>selectolax</code>	HTML email → text, in stage 2	A C-backed HTML parser. Marketing mail is hundreds of kilobytes of HTML; this parses it in milliseconds and stripping is the biggest cost saving in the pipeline.	BeautifulSoup (pure Python, many times slower on large HTML), lxml (heavier dependency, more to install).
<code>email-reply-parser</code>	Cutting the quoted previous conversation and signature, in stage 2	Purpose-built for exactly this; published measurement puts the reduction at ~85% of tokens. One real message went from 156,130 bytes to 5,399 characters.	Hand-written "On ... wrote:" regexes, which break on every mail client's variation.
<code>icalendar</code>	Reading <code>.ics</code> invites, in stage 5	The standard parser for the calendar format. The invite is data: start, end, summary, attendees, read without interpretation.	A model reading the invite text, which can only be as right as the file and can be wrong.
<code>dateparser</code>	Finding date phrases in text, in stage 7	Finds "next Tuesday at 3" in prose, in many languages. Good at the phrases it finds; bad at deciding what counts as a phrase — so our own <i>looks-like-a-date</i> filter and a relative-phrase pattern sit around it.	A model doing date arithmetic: encoders score below chance on it. Production adds Heidelberg and duckling for looser phrasing.
<code>factur-x</code>	Structured invoice XML embedded in PDFs (Factur-X / ZUGFeRD / XRechnung), in stage 5	The embedded-file table in a PDF is fiddly; this library reads it properly. In the POC the embedded XML is detected and flagged; full parsing is the library's job in production.	OCR or a model over the PDF's text, both of which guess at numbers the file already states.
<code>email (stdlib) + zipfile</code>	MIME, headers, <code>.pkpass</code>	Standard library, no dependency.	—

E • Models and machine learning

transformers + torch — the two XLM-RoBERTa models

WHAT

Hugging Face `transformers` loads and runs the models; PyTorch is the numerical engine underneath. Two fine-tuned copies of `xlm-roberta-base` (278M parameters, MIT licence, multilingual) are on disk: the stage-8 classifier in `classifier/` (547 MB) and the stage-10 name finder in `p11_model/` (1.1 GB).

WHY XLM-ROBERTA-BASE

Multilingual — the market is the US, Canada and the UK but mail is not only English, and a test mailbox here contains Urdu. The right size to run on a laptop CPU in ~110 ms. And stable: `mdeberta-v3-base` was tried first, trained fine for one epoch and then collapsed to a single class with the loss going to NaN. XLM-R is the same size and better behaved.

WHY TWO COPIES, NOT ONE

Different heads: sequence classification (one label per email) versus token classification (one label per word). Same base, two jobs.

LOADED ONCE

Both are loaded once per process and held in memory. Loading a 1 GB model per email would be slower than the API it replaces.

WHY NOT A 7B OPEN MODEL

Sir's requirement was 10–100M parameters, CPU-runnable. 278M is already above that band, which is why ONNX 8-bit shrinking is in the plan; a 7B model would need a GPU per worker.

lightgbm — the reply gate

WHAT

Gradient-boosted decision trees. 300 trees over 17 features. Under 1 MB on disk; trains in 0.24 s on 5,895 rows.

WHY TREES AND NOT A NEURAL NETWORK

The features are tabular (counts, rates, booleans, hour of day). Trees are the right tool for tabular data, train instantly per user, are cheap to retrain weekly, and give feature importance for free. A neural net would be slower to train, harder to explain and no better on seventeen numbers.

WHY PER PERSON

"Will Yahya reply to this" has nothing to do with what Sara does; a shared model averages away the signal. It is also a privacy requirement: Google's terms forbid pooling many users' mail into one model.

HONEST NUMBER

AUC 0.684 against a published ceiling of 0.72–0.79, which is why it is blended with hand-set weights rather than trusted alone.

scikit-learn + numpy

`TfidfVectorizer` turns the person's past replies into 384-dimension vectors for stage 19's "find the four most similar replies", compared with a dot product in numpy; `roc_auc_score` evaluates the gate. Production swaps TF-IDF for sentence embeddings and the numpy search for pgvector; the code is written so that swap is one class.

google-genai — Gemini

WHAT

Google's client for Gemini. `gemini-flash-latest` first, with `gemini-2.5-flash`, `gemini-flash-lite-latest` and `gemini-2.5-flash-lite` as fallbacks when the free tier is busy. One client per process.

USED FOR

Writing the draft (stage 11); reading an email only when our classifier's confidence is under 0.65 (stage 6); picking which date phrase is the real deadline; the chat's multiple-choice interpreter when rules do not match; the eight one-line summaries in the brief; and, during training, labelling ten emails per request as the teacher.

WHY GEMINI IN THE POC

A free tier that is enough to prove the flow. Production chooses between Claude and Gemini by blind comparison on real drafts, not by benchmark.

WHAT IT NEVER SEES

Anything that did not pass through stage 10. Names, numbers and addresses are placeholders; the text is cut to 3,000 characters.

datasets · accelerate · sentencepiece · protobuf — training only

`datasets` downloads `ai4privacy/pii-masking-200k`; `accelerate` drives the Hugging Face `Trainer`; `sentencepiece` and `protobuf` are the XLM-R tokenizer's dependencies. None of these run in production.

Kaggle's free T4 GPU

Fine-tuning a 278M model on a laptop CPU takes 20–40 minutes; on Kaggle's free T4 it takes about three. `kaggle_train.py` and `kaggle_train_pii.py` are notebooks-as-files: upload the data, paste the file, run, download the folder. Two hard-won notes live in the file: use the T4, not the P100 (compute capability 6.0 is below what Kaggle's PyTorch build supports), and restrict to one GPU, because the Trainer's DataParallel over two cards fails on 199 examples.

Planned: ONNX Runtime, 8-bit

`shrink_model.py` exports the classifier to ONNX and quantises to 8 bits: ~1.1 GB → ~280 MB, measured elsewhere at roughly 14× faster on CPU, and production never installs PyTorch, taking the container from gigabytes to megabytes. Written; not yet run.

F • Storage

SQLite now, Postgres later

One file, `mailbox.db`, nine tables (see Document 1, Part D). Chosen because it is in the standard library, needs no server, and every query in the POC is small. Production is Postgres with the same schema and pgvector for the style vectors. No query relies on anything SQLite-specific, so the swap is a connection string and a migration.

Why no separate vector database

Stage 19 needs "the four past replies most like this situation": thousands of vectors per user, a few searches a day. Postgres with pgvector does that. Adding Pinecone, Qdrant or Weaviate would mean running a second system for a job the first one handles, and a second place user text would live.

JSON files, and why they are not tables

<code>applied_labels.json</code>	Every label the sorter or watcher applied, with which door applied it. Read by <code>sort_mailbox.py --undo</code> and by the IMAP label-removal check. A file because it must survive a database reset and be readable by eye.
<code>user_sensitive.json</code>	Domains the Owner marked private. Read fresh on every call to stage 3 so a mark takes effect at once.
<code>gate_model.json</code>	Metadata beside the LightGBM model: feature order, AUC, sizes, when trained. The feature order is what makes the model loadable safely.
<code>labelled.jsonl</code> · <code>labels.txt</code> · <code>to_label.txt</code>	The classifier's training data and the hand-labelling round trip. Local only; delete when labelling is done because whoever labels reads the content.

prompts.md — prompts as a document

Every instruction the system gives any model is in one Markdown file with six sections: `classify`, `pick_deadline`, `draft_reply`, `read_command`, `clarify`, `summarise_for_brief`. `pipeline/prompts.py` loads it. Wording is a product decision; sir can change how the system speaks without a developer, the prompts can be reviewed and versioned, and "what are you telling the AI?" has one honest answer.

G · Surfaces: web and terminal

http.server + hand-written HTML and CSS

WHAT

A single Python file serves four routes: the list, one message, "how it works", and a favicon no-op. The CSS is a small token set with a dark-mode media query. No JavaScript framework, no build step, no npm.

WHY

A POC that needs twenty minutes of setup before anyone can look at it does not get looked at. And the page only *reads*: it renders decisions already in the database, so there is no second copy of the logic that could disagree with the pipeline.

WHAT IT IS NOT

A production web server. One user, localhost, no auth. The production view is the same HTML behind a real server, or the WhatsApp channel the design specifies.

The terminal

Every script prints with ANSI colours (a small `C` dictionary of escape codes) and `os.system("")` on Windows to switch them on. Output is designed to be pasted into a chat and read, which is how most of the debugging in this project happened. The `-X utf8` flag is needed on Windows consoles for subjects containing characters outside cp1252.

H · Secrets, configuration and flags

ITEM	WHERE	NOTE
IMAP_USER · IMAP_PASSWORD	Windows user environment variables via <code>setx</code>	Only visible in terminals opened afterwards. The password is a Google app password, never the account password. Never printed; the setup check prints <code>SET</code> or <code>MISSING</code> .
GEMINI_API_KEY	environment variable	Optional. Without it stages 8 (our model only), 9, 10, 13, 14 still run; 11 is skipped and says so. A key was once pasted into a chat and must be treated as exposed until its replacement is confirmed.
credentials.json · token.json	beside the code	The OAuth client and the user's grant. <code>token.json</code> present means the API door is the default.
--imap · --api	any script	Force a door. The flag wins over the environment.
--draft · --drafts	agent, watch	Write a reply; put it in Gmail Drafts instead of asking to send.
--watch · --port	web.py	Run the agent behind the page; change the port.
--write · --send · -- no-connections	the proofs	Enable the tests that change something, send a mail to yourself, skip the connection probe.

I • What is deliberately not used

NOT USED	WHY NOT
Our own SMTP server or mail client	Sir's hard constraint. We are a layer on top of mailboxes people already have; every company that tried to own the whole mail experience here has died or been sold. Sending goes through the provider's own SMTP or API.
The full Gmail scope	https://mail.google.com/ permits permanent delete. Not asking is stronger than promising not to.
A vector database	Postgres with pgvector does the one vector job the system has.
An agent framework / MCP server	They collapse instruction text and email text into one model loop, which is exactly the separation the safety of the system rests on. Sir refrained from it explicitly.
A web framework or JS build	Nothing to install; nothing to break; the page only reads.
A model for date arithmetic, sensitivity, or the approval gate	Models score below chance on date arithmetic; the gate must be un-arguable; sensitivity must be certain, not probable.
Summarising the whole mailbox	Thousands of paid calls to produce a wall nobody reads. The brief summarises at most eight things that survived the pipeline.
Body-link unsubscribe	Clicking through a stranger's page automatically executes whatever that page asks. Only RFC 8058 one-click is done.

J • The production swap list

What changes between this proof of concept and a product, and what does not. The pipeline, the schema, the gate and the connector interface do not change.

COMPONENT	POC	PRODUCTION	WHAT HAS TO CHANGE IN CODE
Stage 8 reader	XLM-R fine-tuned on 199 examples; Gemini when unsure	Same model, ONNX 8-bit, on a few hundred careful labels; escalation stays	<code>local_classifier._load()</code> loads ONNX instead of PyTorch
Stage 9 gate	LightGBM per mailbox, blended with hand weights	Same, retrained weekly per user, weights learned	A scheduler; nothing in the stage
Stage 10 redaction	Regex + XLM-R token model	Presidio wrapping a fine-tuned DeBERTa-v3-small (44M)	The model behind <code>local_pii.find()</code>
Stage 11 writer	Gemini free tier	Claude or Gemini by blind comparison	The client in <code>model._generate()</code>
Stage 19 vectors	TF-IDF blobs in SQLite, numpy search	Sentence embeddings in pgvector	The <code>Embedder</code> class and one query
Storage	SQLite file	Postgres	Connection string, migration
Trigger	Poll every 15–20 s	Gmail push via Pub/Sub; IMAP IDLE per worker	<code>watch.py</code> and the web watcher; the pipeline is untouched
Channel to the person	Terminal chat, local web page	WhatsApp; the web view served properly	A new surface using the same <code>user_input.read()</code> and gate
Doors	Gmail API, IMAP	+ Microsoft Graph, JMAP, Zoho	One new file each under <code>connectors/</code>
Date finding	dateparser + our filters	+ HeidelTime, duckling	Inside <code>stage07_dates.find_candidates()</code>